

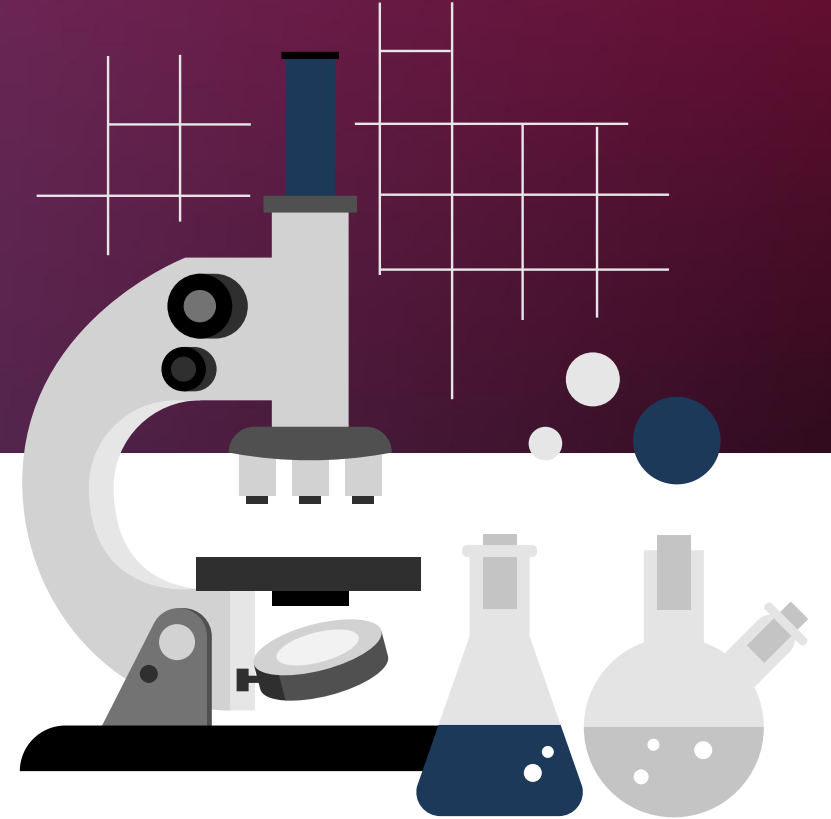
# Analyse de données

Mathieu SEURIN  
mathieu.seurin@devinci.fr

et

Nancy CHENDEB  
nancy.chendeb@devinci.fr

## CM5 : ARBRE DE DÉCISION



# \ RAPPEL NOTATION JEU DE DONNÉES

|           | $x_0$ | $x_1$       |
|-----------|-------|-------------|
|           | Pluie | Température |
| $x^{(0)}$ | Oui   | 0           |
|           | Non   | 30          |
|           | Oui   | 27          |
|           | Non   | 0           |
| $x^{(5)}$ | Non   | 20          |

$X$

| Type de haut |           |
|--------------|-----------|
| Doudoune     | $y^{(0)}$ |
| T-shirt      |           |
| K-Way        |           |
| Doudoune     |           |
| T-shirt      | $y^{(n)}$ |

$y$

$$n = 5$$

*Nombre d'exemples*

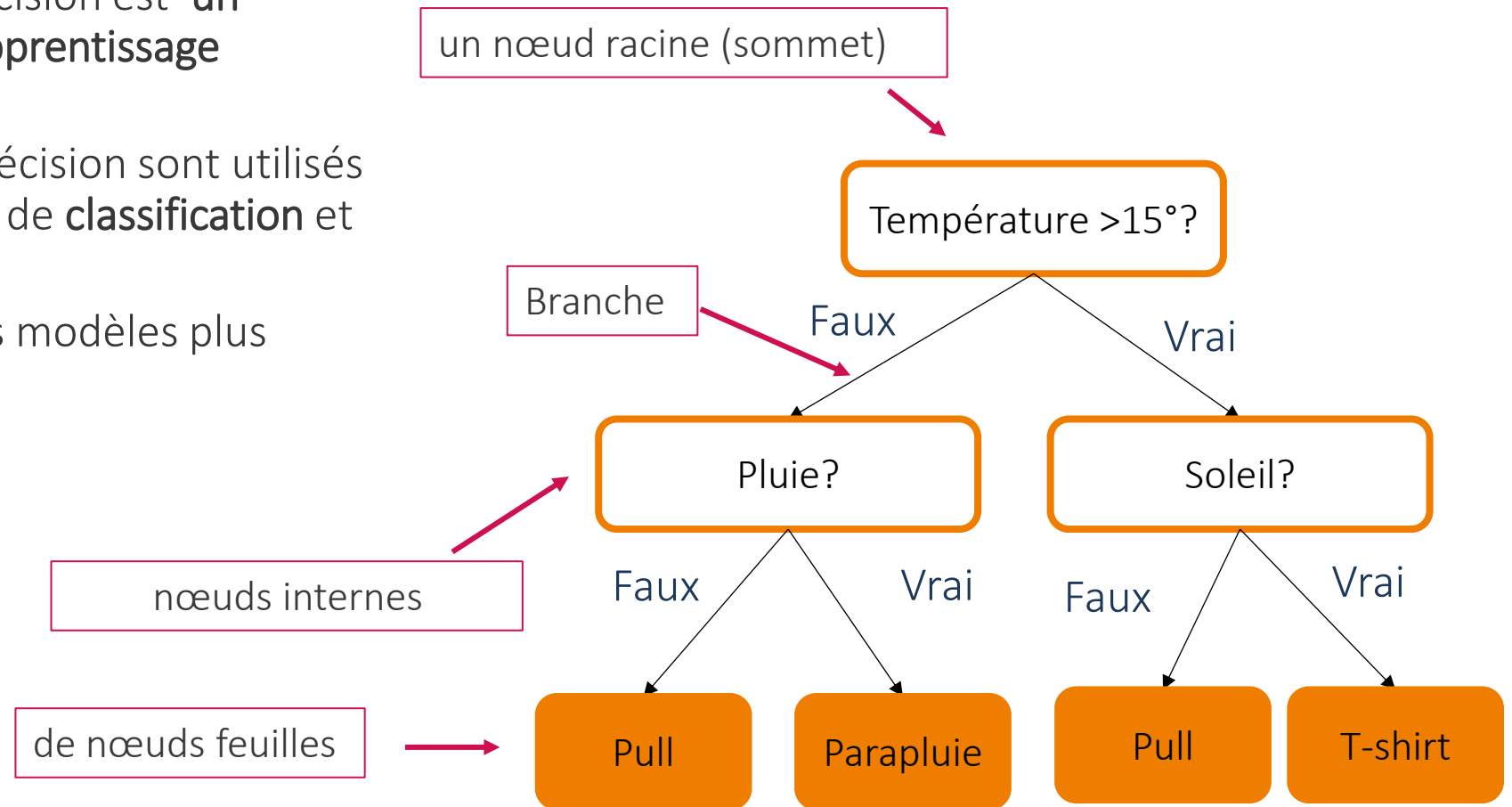
$$k = 2$$

Nombre de caractéristiques

N'oubliez pas la séparation train/test

# QU'EST CE QU'UN ARBRE DE DÉCISION?

- Un arbre de décision est un **algorithme d'apprentissage supervisé**.
- Les arbres de décision sont utilisés pour les tâches de **classification** et de **régression**
- Ce sont des modèles plus interprétables.



# TERMINOLOGIE

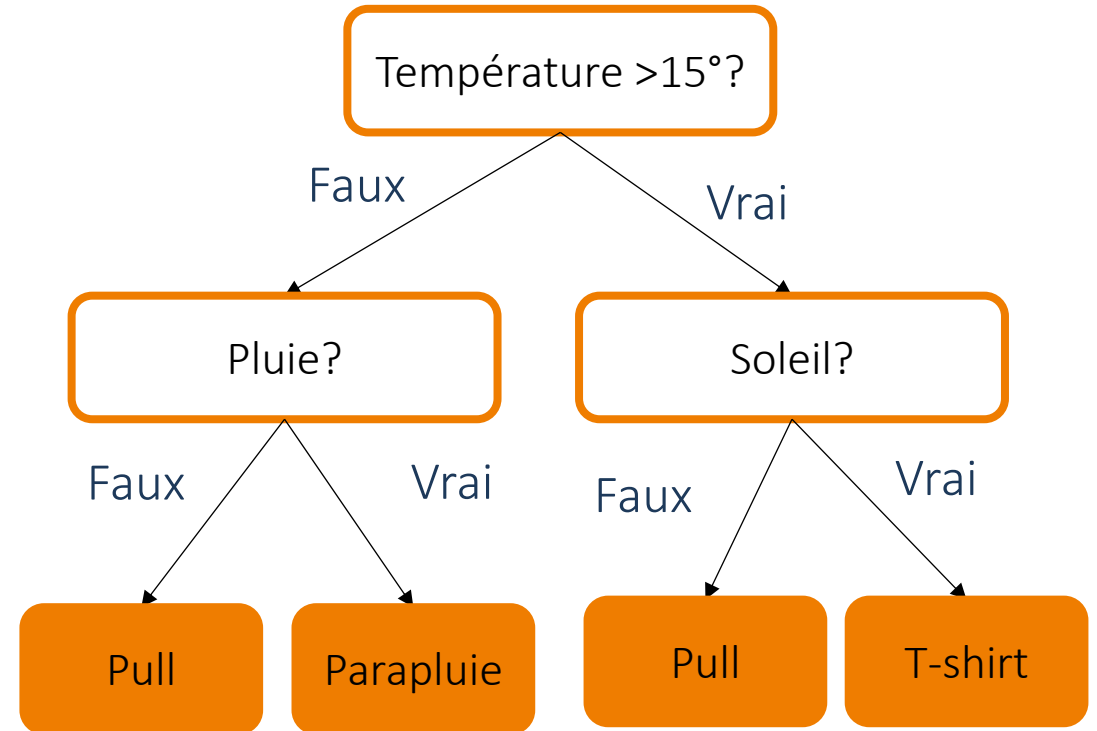
**Nœud racine/sommet** : nœud initial au début d'un arbre de décision

**Nœuds de décision** : Nœuds résultant du fractionnement (split) des nœuds racines. Ces nœuds sont des conditions intermédiaires

**Nœuds feuilles ou terminaux** : nœuds où une division ultérieure n'est pas possible, indiquant la classification ou le résultat final.

**Sous-arbre** : sous-section d'un arbre de décision.

**Nœud parent et enfant** : Un nœud divisé en sous-nœuds est appelé nœud parent, et les sous-nœuds qui en émergent sont appelés nœuds enfants (ou fils).



# PRÉDICTION AVEC UN ARBRE

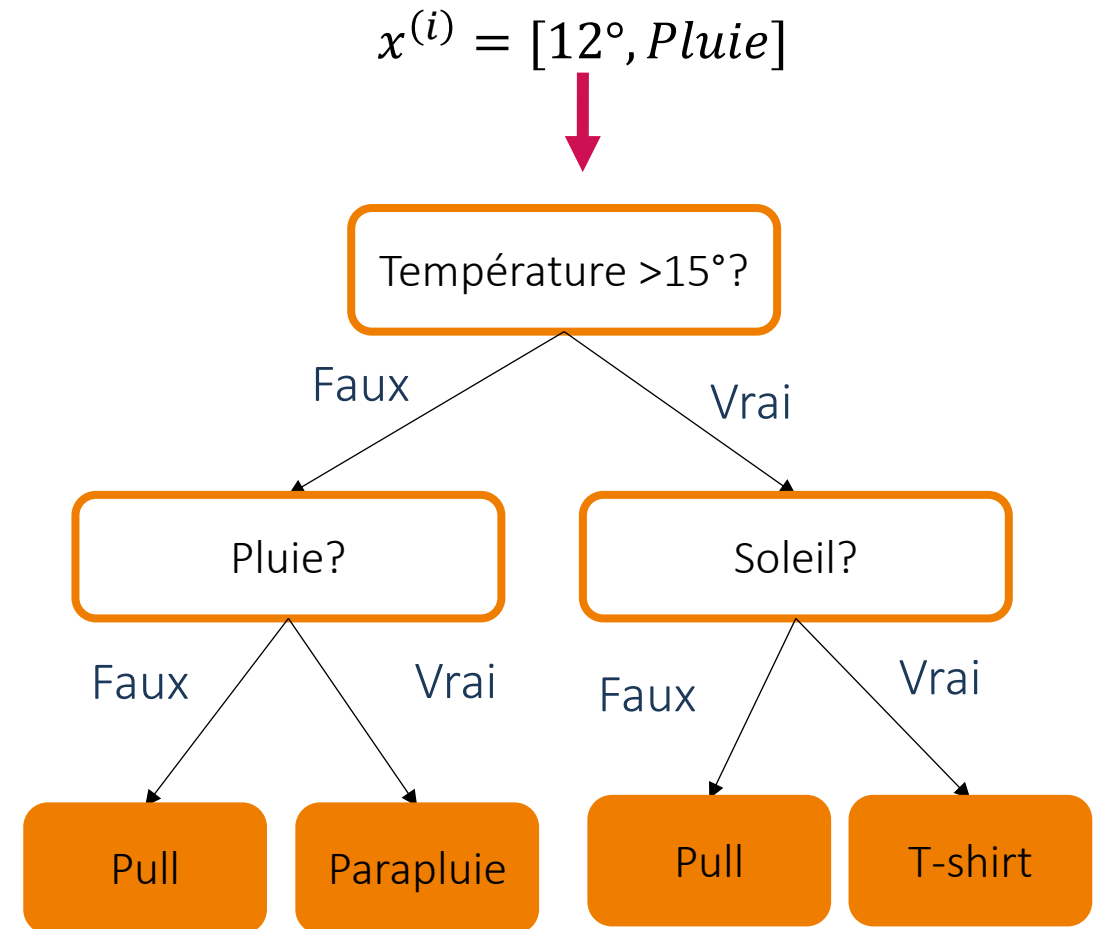
Les arbres de décision commencent par la racine en haut, puis cette racine est divisée en plusieurs nœuds.

C'est un ensemble d'instructions if-else

Si la condition est vraie, il passe au nœud suivant attaché à cette décision.

Sinon, il passe au nœud suivant attaché à la décision contraire

Et ainsi de suite jusqu'à prendre décision...



# LES ARBRES DE DÉCISION

- Principalement pour de la classification
- Gère à la fois des caractéristiques catégorielles et numériques.
- Modèle très expressif
- Très bon sur données tabulaires
- Peu interprétable (quand il y a peu de variables et peu de nœud)
- Modèle à la base de beaucoup de modèles qui gagnent des compétitions kaggle (XGBoost, LightGBM)
- Optimisation très différente de ce qu'on a vu jusqu'à présent

Comment construire un arbre de décision ?

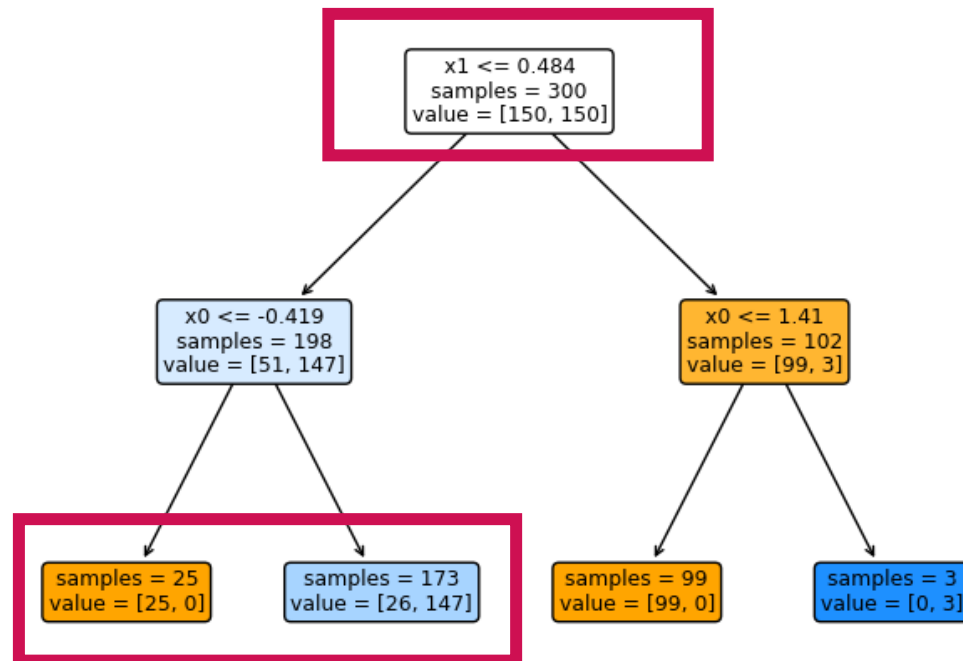
# FRONTIÈRE DE DÉCISION

$X$  jeu de données

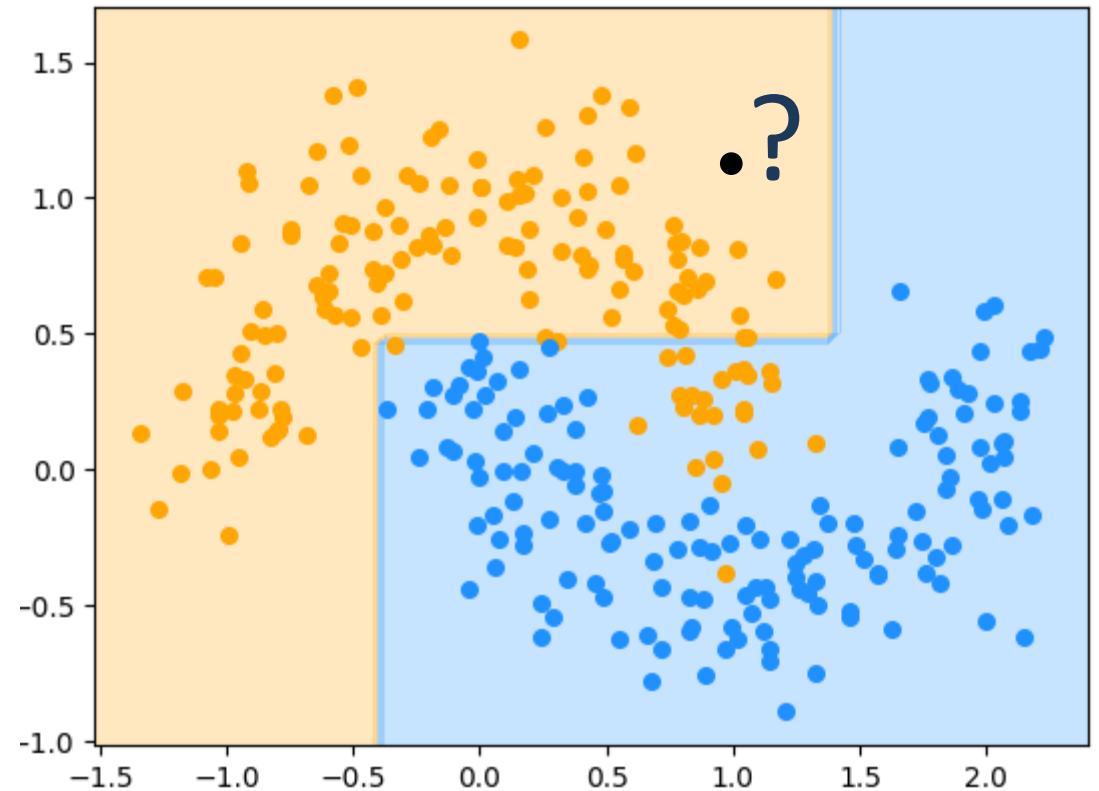
$y \in \{0, 1\}$

Problème à deux classes pour le moment

Nœud = une décision



Feuilles = assignation à une catégorie  
(classe majoritaire dans cette feuille)



# \ FONCTION DE COÛT

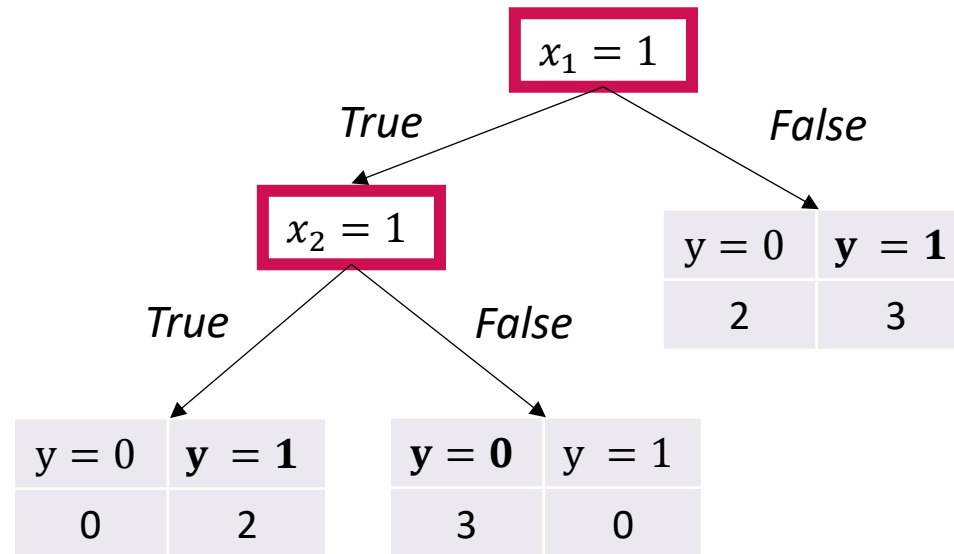
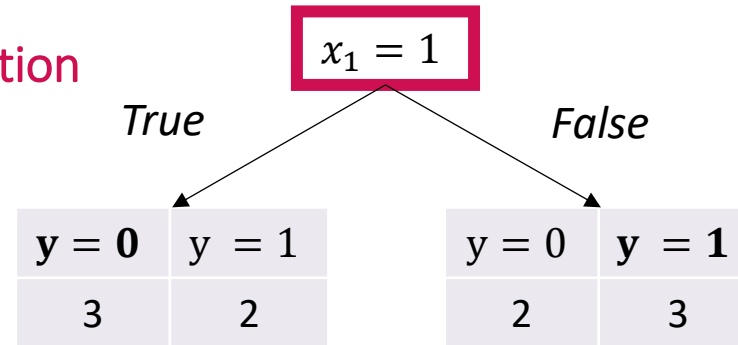
1<sup>er</sup> idée de coût : %de bonne classification  
au niveau des feuilles

Arbre 1 :  $3/10 + 3/10 = 60\%$   
d'accuracy

Arbre 2 :  $2/10 + 3/10 + 3/10 =$   
80% d'accuracy

Meilleur!

**Remarque :** une fois dans une  
partie de l'arbre, les  
changements n'affectent pas  
la précision de l'autre branche



| $x_1$ | $y$ |
|-------|-----|
| 0     | 1   |
| 1     | 1   |
| 0     | 1   |
| 1     | 1   |
| 0     | 1   |
| 1     | 0   |
| 0     | 0   |
| 1     | 0   |
| 0     | 0   |
| 1     | 0   |

# \ SUFFISANT ?

Formalisation du cout :

$$\frac{1}{n} \sum_{i=0}^n \mathbb{1}_{\{y \neq \hat{y}\}}$$

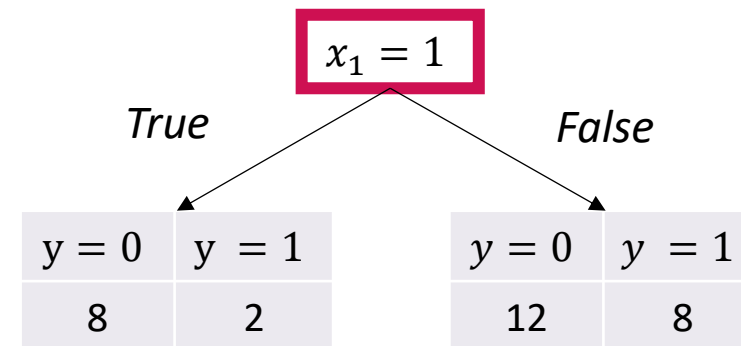
1<sup>er</sup> arbre : 60% de précision

2<sup>e</sup> arbre :  $8/30 + 12/30 = 60\%$

60% de précision et arbre plus complexe mais, distribution plus « marquée » pour la feuille 1

Comment formaliser une distribution plus « marquée » ?

| $y = 0$ | $y = 1$ |
|---------|---------|
| 20      | 10      |



# UN PEU DE THÉORIE DE L'INFORMATION

L'entropie est un concept qui revient dans beaucoup de domaine, et à pleins d'interprétations, pleins de manière d'être décrite (Mesure de dispersion, mesure du chaos etc...)

Dans le cadre des statistiques, on va parler d'entropie d'une distribution notée  $H$ .

$$H(Y) = - \sum_{i \in \{y\}} p(y = i) * \log_2(p(y = i))$$

L'entropie mesure l'impureté d'un nœud. L'impureté est le degré de hasard ; cela indique à quel point nos données sont aléatoires.

| $y = 0$ | $y = 1$ | $H(Y)$ |
|---------|---------|--------|
| 5       | 5       | 1      |
| 7       | 3       | 0.88   |
| 10      | 0       | 0      |

Une entropie forte = proche de la loi uniforme

Une entropie faible = distribution moins équilibrée

Entropie nulle = Division **pure**

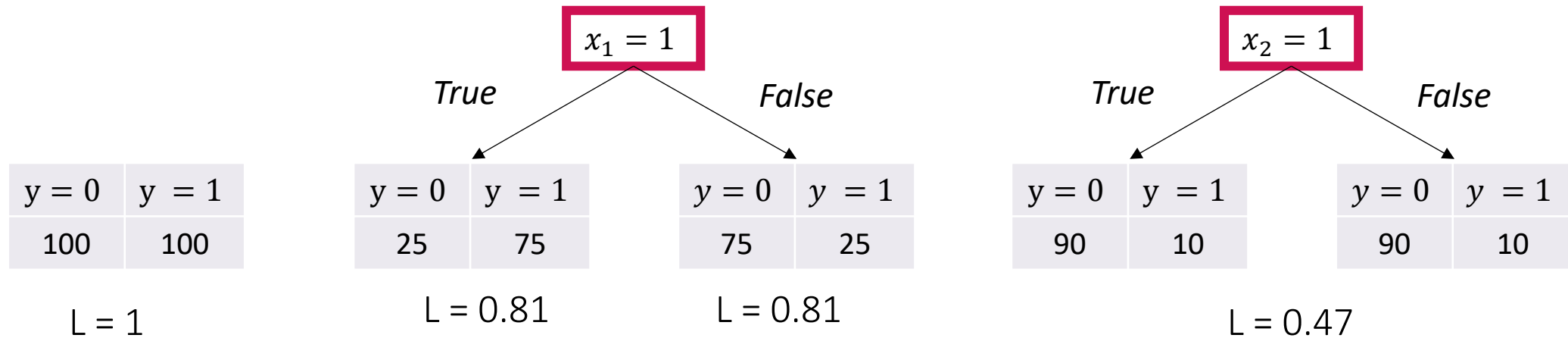
# \ FONCTION DE COÛT POUR UN ARBRE DE DÉCISION

$A$  : un arbre

$$L(A, X, y) = \frac{1}{n} \sum_{i=1}^n H(A.Feuille(x^{(i)}, y^{(i)}))$$

avec  $Feuille$ , la fonction qui renvoie la distribution des  $y$ s à la feuille où tombe  $x^{(i)}$

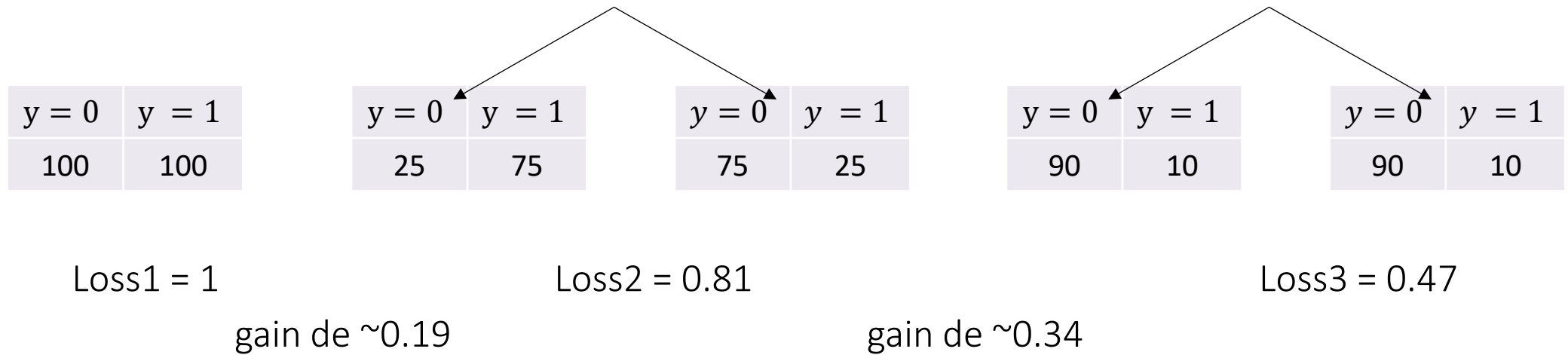
- Et  $H(Y) = - \sum_{i \in \{y\}} p(y = i) * \log_2(p(y = i))$



# COÛT ET GAIN D'INFORMATION

$$L(A, X, y) = \frac{1}{n} \sum_{i=1}^n H(A.Feuille(x^{(i)}, y^{(i)}))$$

$$\text{Et } H(Y) = - \sum_{i \in \{y\}} p(y = i) * \log_2(p(y = i))$$



On parle de **Gain d'Information** (diminution de l'entropie)

# \ OPTIMISATION DE LA FONCTION DE COÛT

## Sur quelle caractéristique on fait le split ?

- Première idée :
  - Recherche greedy du meilleur split (on parcourt toutes les features et on test tous les seuils)
  - Recherche à une profondeur de 1
- Algorithme :
  - Calcul du gain d'information d'ajouter un split pour chaque feature
  - Sélectionner le split qui apporte le plus de gain
  - Trier les données dans cette nouvelle feuille
  - Répéter jusqu'à un critère d'arrêt (à définir)

# TROUVER LE MEILLEUR SPLIT

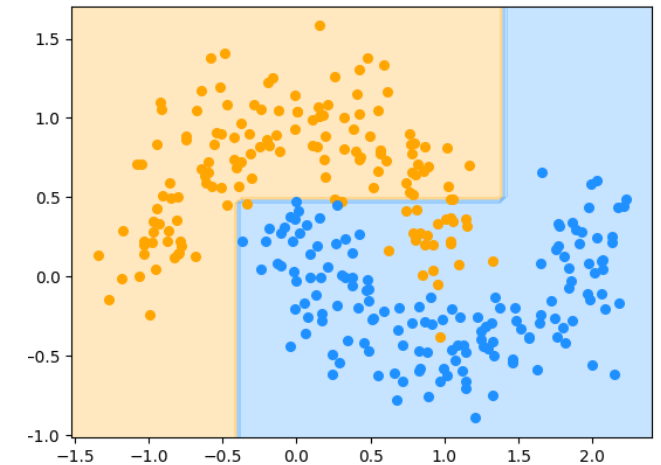
```

def meilleur_features(arbre, dataset):
    information_gains = []
    for feat in range(#features):
        information_gains = gain_pour_feature(arbre, dataset, feat)

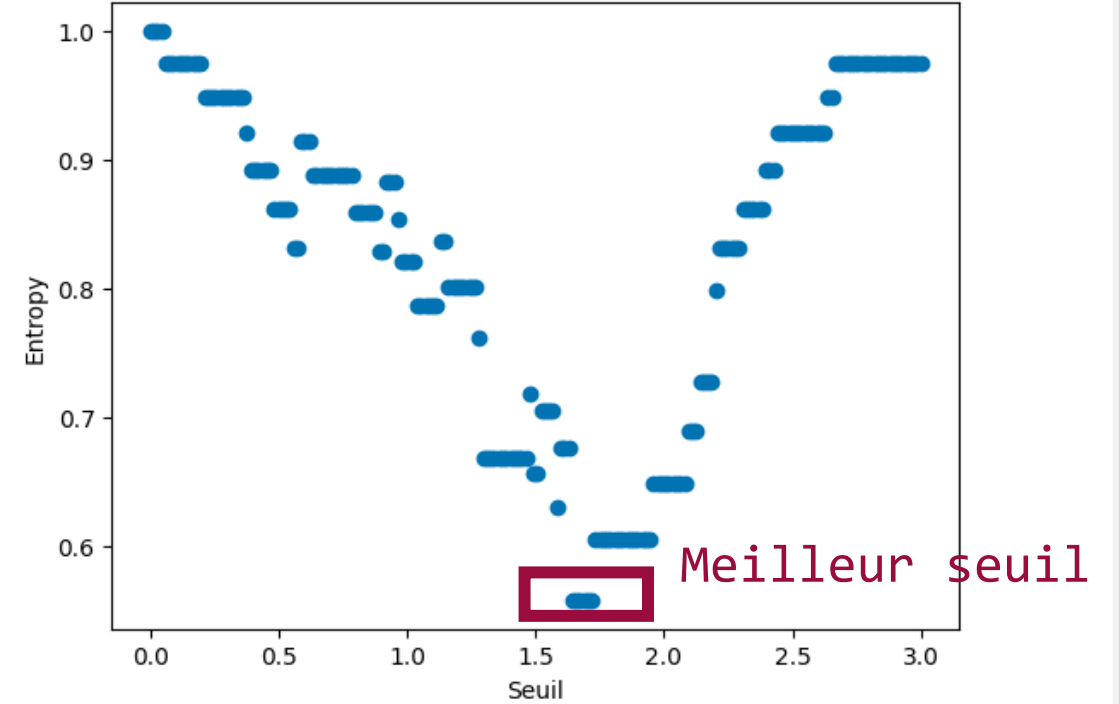
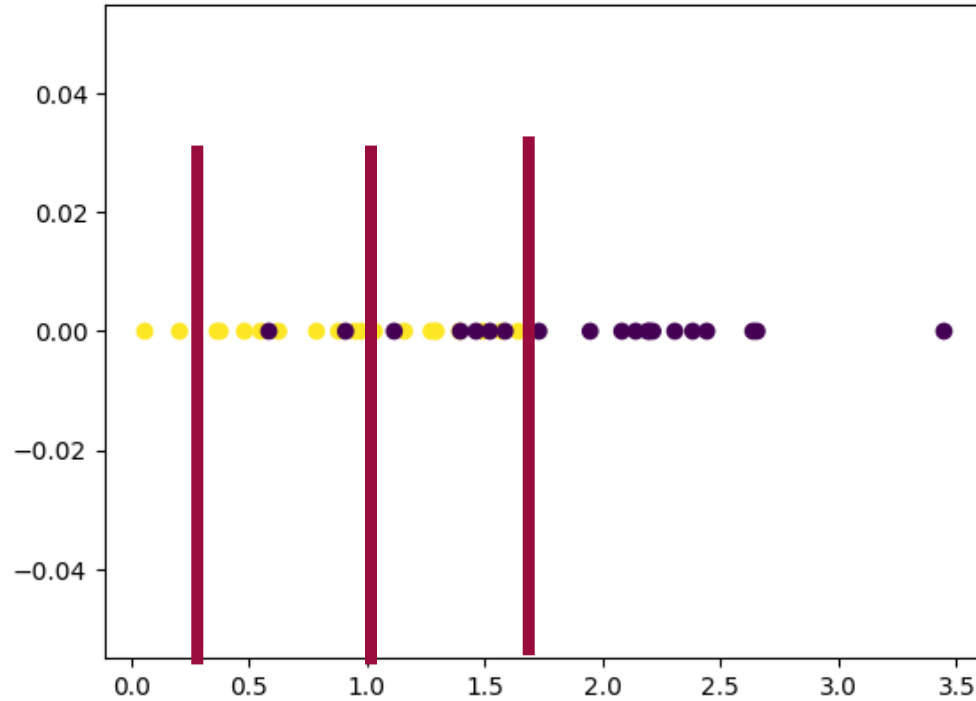
    meilleur_feat = argmax(information_gains)
    meilleur_info_gain = information_gains[meilleur_feat]
    return meilleur_feat

def gain_pour_feature(arbre, dataset, feat):
    return entropy(arbre, dataset) - split_entropy(arbre, dataset, feat)

def split_entropy(arbre, dataset, feat):
    somme_entropy = 0
    for splitted_distrib in SplitByFeaturesValues(arbre, S, feat):
        somme_entropy += entropy(splitted_distrib) * len(splitted_distrib)
  
```



# MEILLEUR SPLIT SUR UNE FEATURE



# QUAND ARRÊTER L'ALGO ?

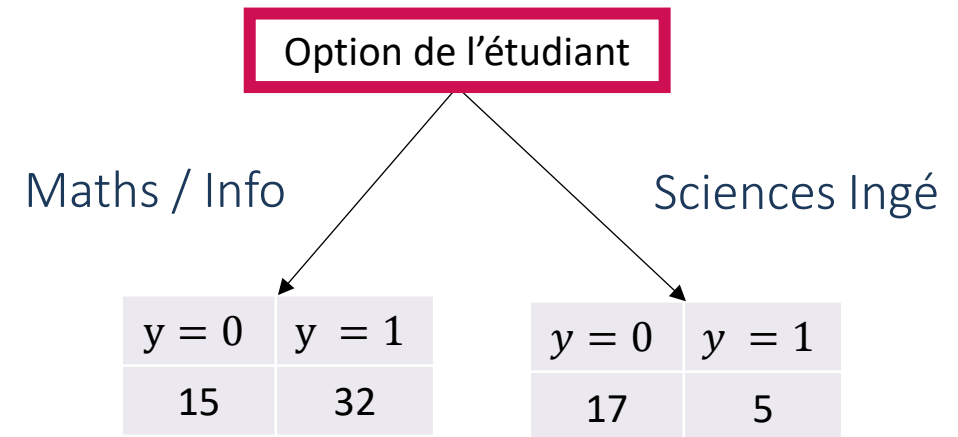
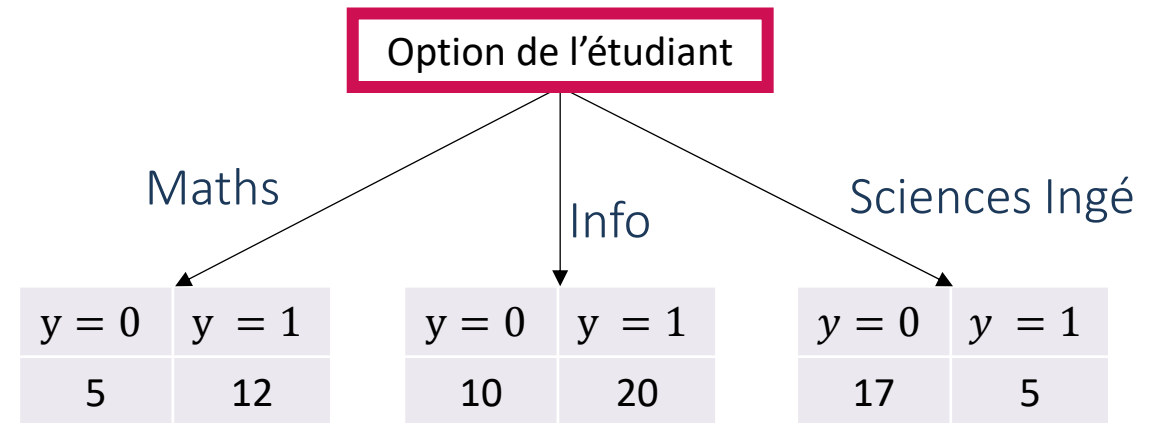
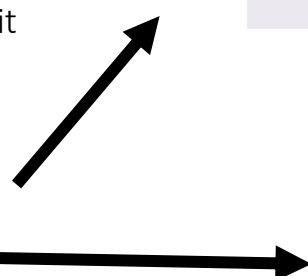
- Critère sur le coût
  - Une feuille est pure
  - Quand un split ne change plus le coût (seuil à définir)
  - Quand il n'y plus de features sur laquelle on peut faire un split (ça n'arrivera pas sur les valeurs continues)
- Critère sur le nombre de sample au niveau de la feuille
  - La feuille contient moins de K samples (avec K un seuil à définir)
- Critère de complexité du modèle
  - Nombre de nœud dépasse un seuil
  - La profondeur de l'arbre dépasse un seuil
  - Pénalité sur la complexité de l'arbre
- Un mix de tout ça !

# TYPE DE DONNÉES DES FEATURES

- Features numériques jusqu'à présent (trouver un seuil etc...)

Mais les arbres peuvent aussi gérer :

- Features booléenne
  - Pas de seuil à calculer pour le split
- Catégorie : Numérique ou Non !
  - Un enfant par catégorie possible
  - 1 vs le reste
  - 1 ensemble vs le reste



Attention : scikit ne permet pas d'utiliser des variables catégorielles

# CONSIDÉRATIONS PRATIQUES

# TROUVER LE MEILLEUR SPLIT POUR CHAQUE FEATURE

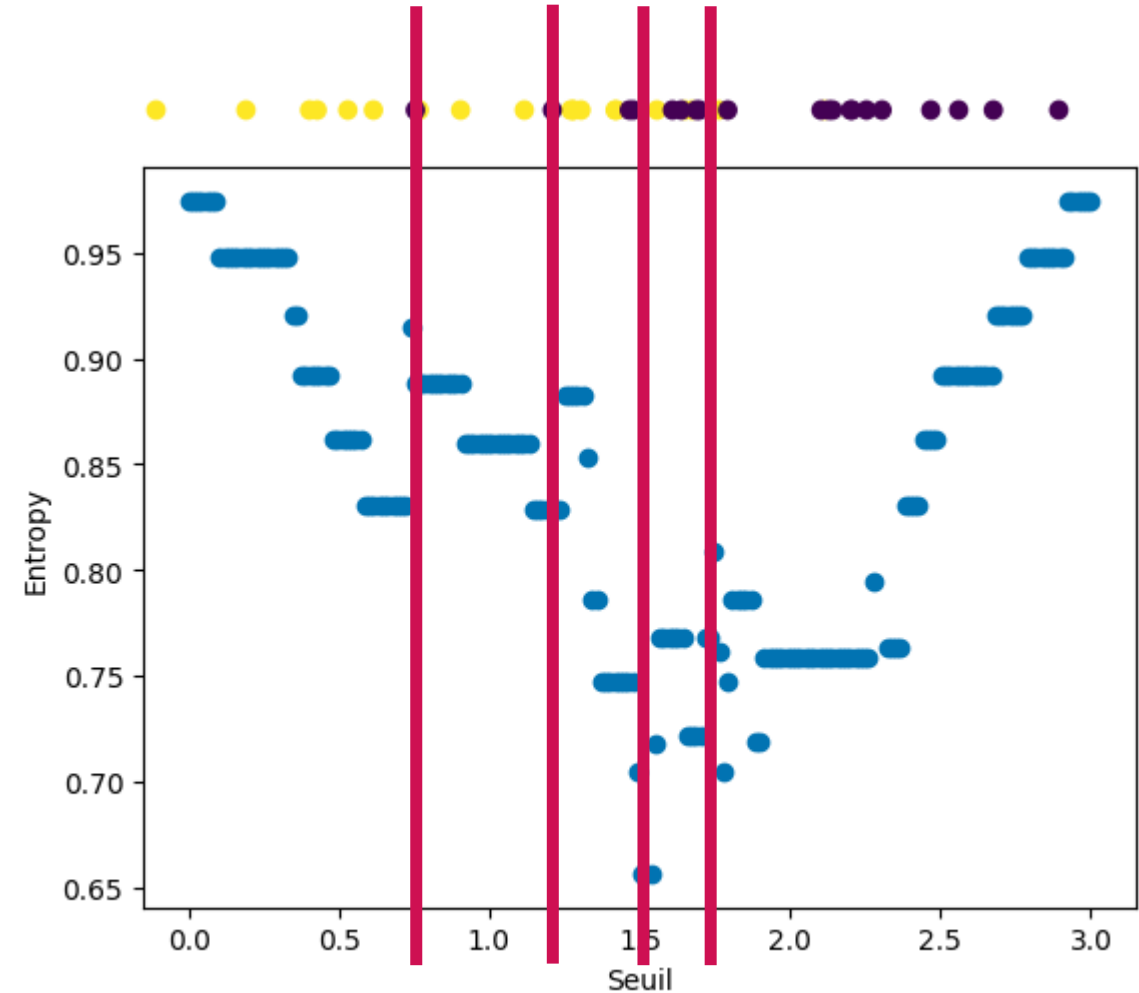
Si on a énormément de features, trouver le meilleur seuil pour toutes peut être chronophage. (calcul de l'entropie pour chaque seuil)

Idée simple : Faire des intervalles (problème de sensibilité)

Idée maligne et plus rapide : Plutôt que de regarder TOUS les seuils, prendre ceux endroits il y a une alternance des deux classes

Idée encore plus simple : Prendre un seuil aléatoire (n'est en fait pas si mauvais si on sample un peu)

On verra ça en TD!



# SORTIE MULTIPLE

Pour le moment, seulement 2 classes, pas très général ...

Changement pour les arbres :

- Chaque feuille stocke k valeurs au lieu de 2 (nombre de classes)
- Changement du critère pour tenir compte de k classes

$$- H(Y) = - \sum_{i \in \{y\}} p(y = i) * \log_k(p(y = i))$$

- Possibilités de garder un seul seuil à chaque nœud
- Tant que l'entropie diminue avec un split, c'est bon!

| $y = 0$ | $y = 1$ | $y = 2$ | $H(Y)$ |
|---------|---------|---------|--------|
| 5       | 5       | 5       | 1      |
| 7       | 3       | 1       | 0.72   |
| 18      | 1       | 1       | 0.35   |
| 0       | 0       | 10      | 0      |

# AUTRE FONCTION DE COÛT : GINI

Pour le moment, on a vu :

- Loss de classification
- Entropy

Le critère utilisé en pratique (notamment par sklearn) est **l'index de Gini** (pureté) :

- $G(node) = \sum_{i=1}^k p_i * (1 - p_i)$  avec  $p_i$  la proba pour un objet d'être dans la classe  $i$  (plus efficace niveau calcul, avec tendances similaires)

Uniforme : Score de 0.5

Plus piquée : Tend vers 0

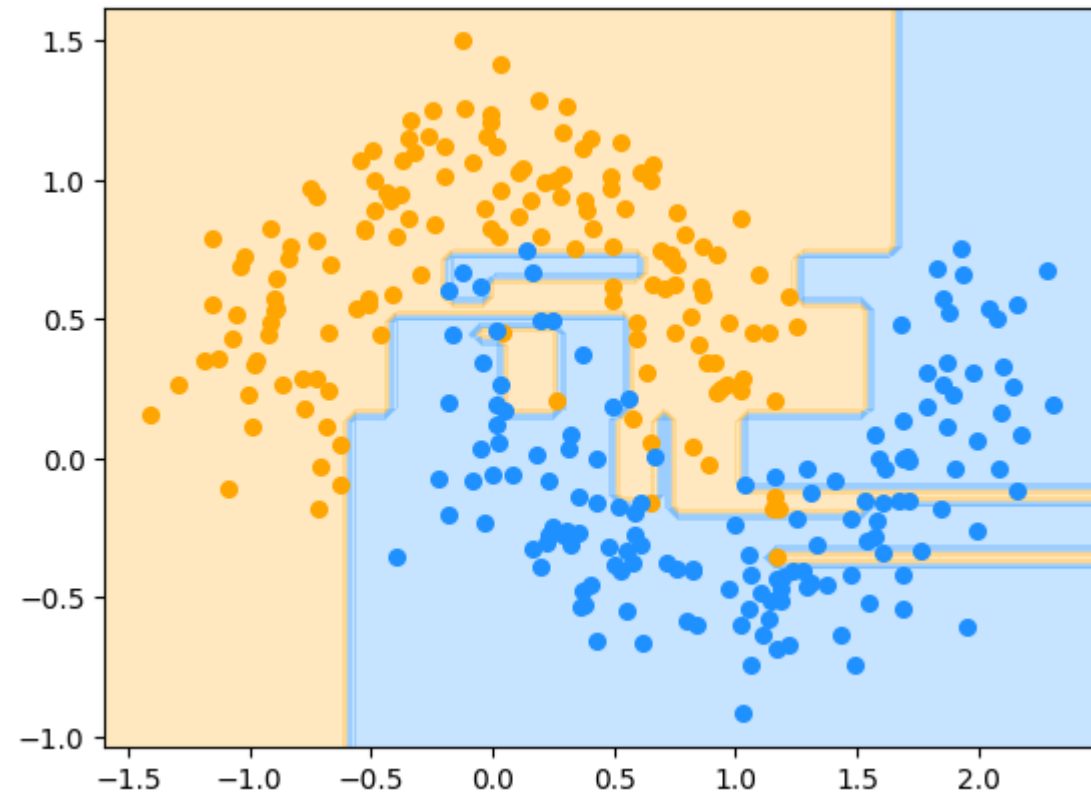
| $y = 0$ | $y = 1$ | $G$  |
|---------|---------|------|
| 5       | 5       | 0.5  |
| 7       | 3       | 0.42 |
| 1       | 9       | 0.18 |
| 10      | 0       | 0    |

# LE SUR-APPRENTISSAGE

L'arbre étant un modèle très expressif,  
le sur-apprentissage risque d'être  
encore plus présent

Deux méthodes de pruning :

- Pre-pruning
- Post-pruning



# **PRUNING**

**Pruning (Élagage)** : processus de suppression ou de réduction de nœuds spécifiques dans un arbre de décision pour éviter le sur-apprentissage et simplifier le modèle.

- L'élagage aide à améliorer les performances de l'arbre de décision en supprimant les nœuds ou sous-nœuds qui ne sont pas significatifs.
- Il supprime aussi les branches qui ont une très faible importance.

Il existe principalement 2 façons de tailler :

**Pre-pruning (Pré-taillage)** – nous pouvons arrêter la croissance de l'arbre plus tôt, ce qui signifie que nous pouvons élaguer/supprimer/couper un nœud s'il a peu d'importance **lors de la croissance** de l'arbre.

**Post-pruning (Post-élagage)** – une fois que notre **arbre a atteint sa profondeur** , nous pouvons commencer à tailler les nœuds en fonction de leur importance.

## \ PRE-PRUNING

Choisir des paramètres pour limiter la croissance :  
(dans sklearn)

- **max\_depth** : Limiter la profondeur max de l'arbre
- **min\_samples\_split** : Limiter le nombre de sample avant de faire un split
- **min\_samples\_leaf** : Après un split, il doit y avoir plus de *min\_samples\_leaf* dans chaque feuille

# POST-PRUNING

Post Pruning, réduire l'arbre a posteriori : ( $L$  la loss entropy/gini)

$$L_{\alpha}(T) = L(T) + \alpha|\tilde{T}|$$

avec  $|\tilde{T}|$  le nombre de nœud terminaux et  $\alpha$  un paramètre de régularisation (comme pour la régression, il équilibre entre la loss d'entropy et la régularisation)

Supprime ensuite les nœuds qui ne change que peu le score de classification.

# CLASSES DÉSÉQUILIBRÉES

Quand les classes sont déséquilibrées :

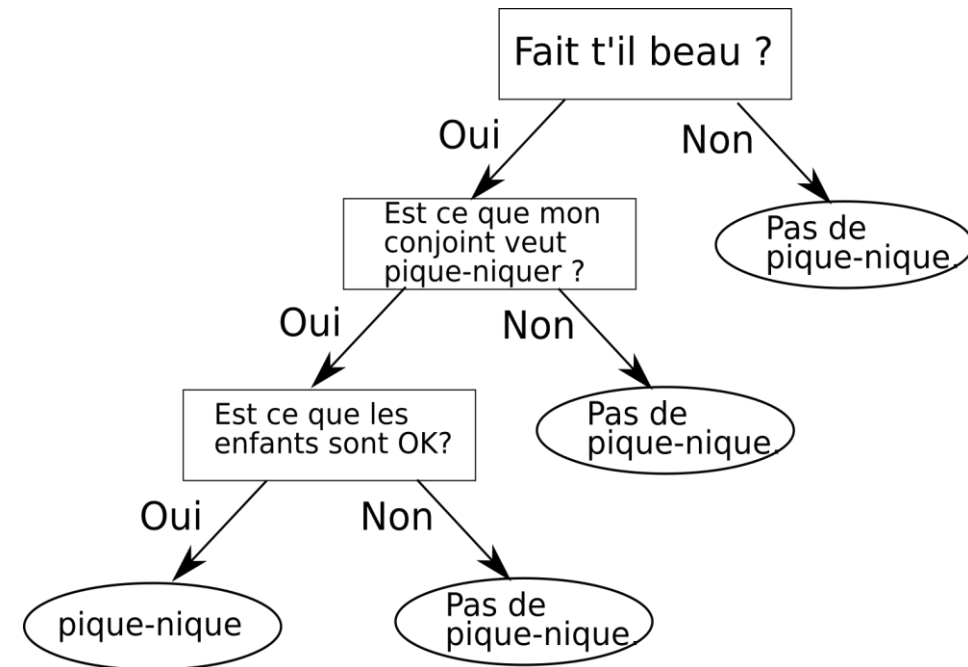
- Problème dans le critère de classification (Ici un algo qui prédit  $y=0$  obtient 90% de précision ...)
- Durant l'optimisation, peut créer des (surtout quand les classes sont fortement déséquilibrées)
  - Exemple : Prendre la classe majoritaire au sein d'une feuille risque => Risque de ne jamais prédire 1
  - Une solution : Mettre un poids plus important aux éléments de la classe minoritaire.

| $y = 0$ | $y = 1$ |
|---------|---------|
| 900     | 100     |

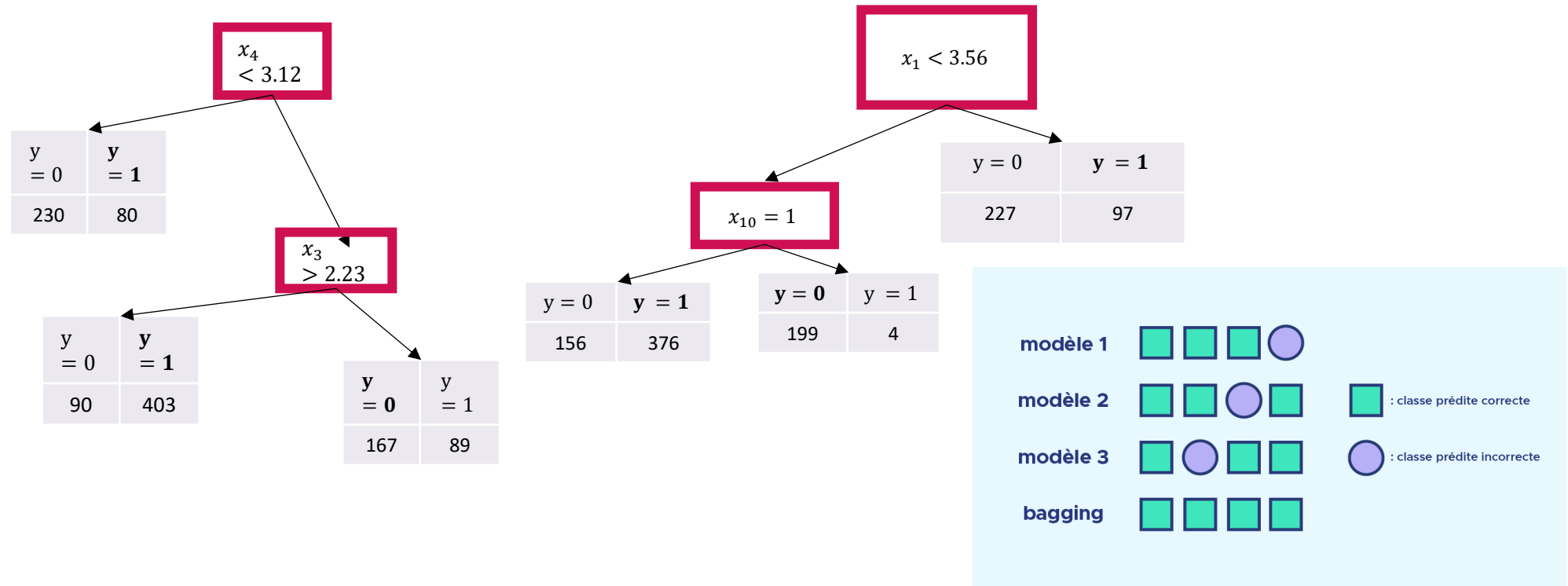
(Pas spécifique aux arbres de décisions)

# INTERPRÉTATION

- Les variables en haut de l'arbre ont le plus d'importance dans la décision
- Permet parfois d'identifier des valeurs intéressantes pour les caractéristiques
- Mais globalement, beaucoup de variables, arbres profonds = Impossible à analyser

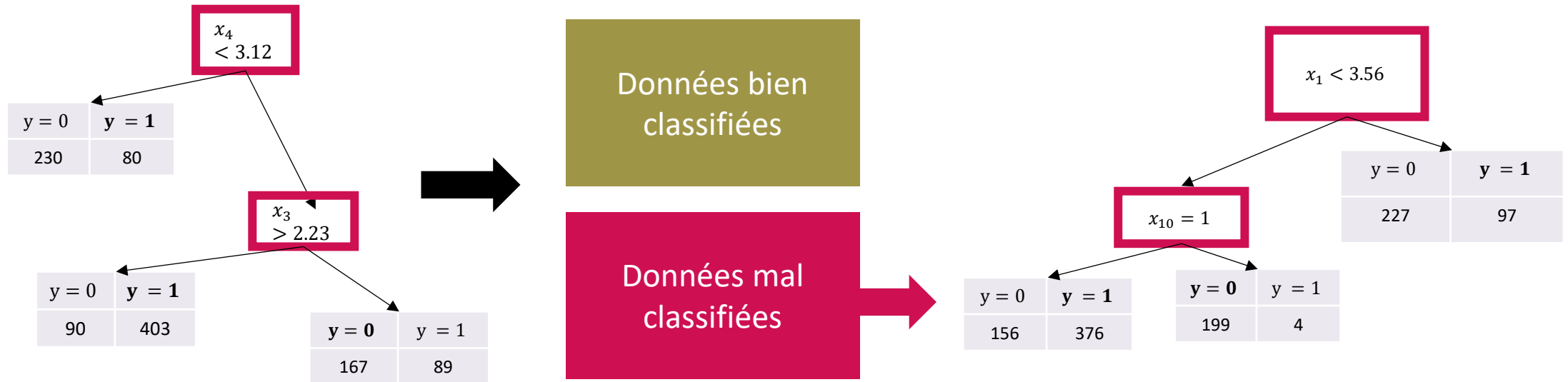


# RANDOM FOREST ET CALCUL EN PARALLÈLE



Random forest = Apprendre plusieurs arbres, bien plus petit et moyenner les décisions (Bagging)  
(Arbres plus petits et en parallèle = super rapide)

# BOOSTING : CONSTRUCTION SEQUENTIELLE



Gradient boosting = Apprendre plusieurs arbres de manières séquentielles :

Les mauvaises classifications sont données à apprendre à un deuxième, puis on répète avec un troisième etc ...

# CONCLUSION

- Modèle très expressif
- Pleins de manières de le construire
- Beaucoup de paramètres à choisir pour la régularisation
- Un peu interprétable (quand il y a peu de variables et peu de nœud)
- Modèle à la base de beaucoup de modèles qui gagnent des compétitions kaggle (avec bagging et boosting)
- Optimisation très différente de ce qu'on a vu jusqu'à présent

# **AVEC PYTHON**

**from** sklearn **import** tree

**from** sklearn.tree **import** DecisionTreeClassifier, DecisionTreeRegressor

**# Entraînement du modèle classifieur**

model = DecisionTreeClassifier()

model.fit(X, y)

**# prédire la classe de l'échantillon [x0,y0]**

model.predict([[x0,y0]])

**# prédire la probabilité d'une classe de l'échantillon [x0,y0]**

model.predict\_proba([[x0,y0]])

**# tracer l'arbre**

tree.plot\_tree(model)

**# Entraînement du modèle regresseur**

model = **DecisionTreeRegressor()**

model.fit(X, y)

